

Statistical Foundations of Machine Translation & LLMs

SPA Direct Reading Program

Mentee: Taeyun Kim

Mentor: Tony Lei



Motivation

Goal of NLP language models:

Predict the next word in a sequence.

Example:

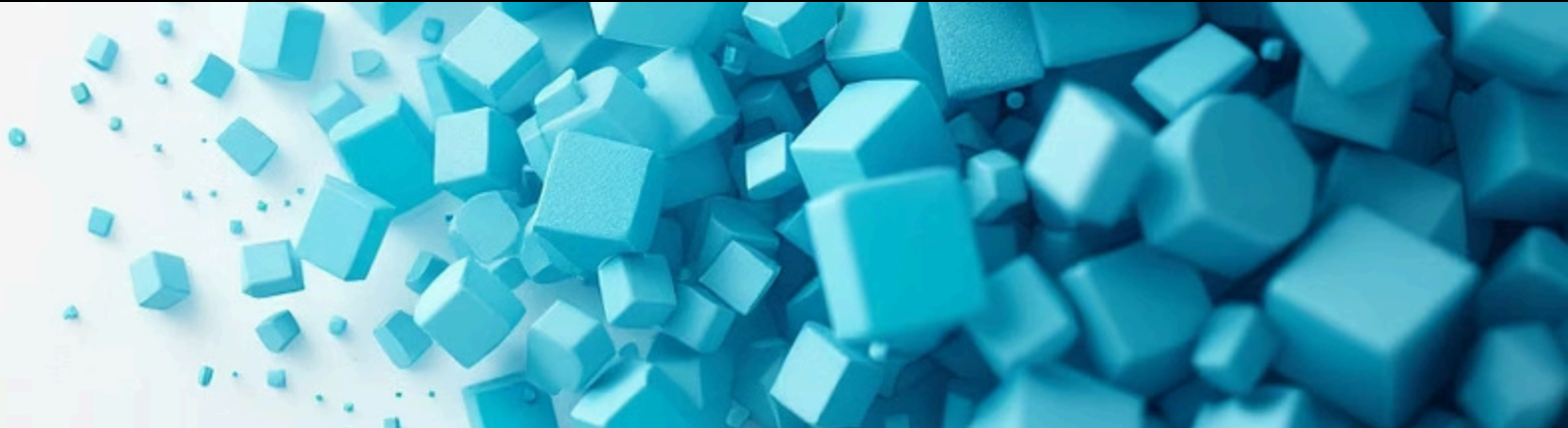
The cat ____

Possible predictions:

- jumped
- ran
- slept

Language models estimate $P(W|H)$ where:

- W = next word
- H = history (previous words)



Words vs Tokens

Words

Natural language units

Example: "The cat jumped"

Tokens

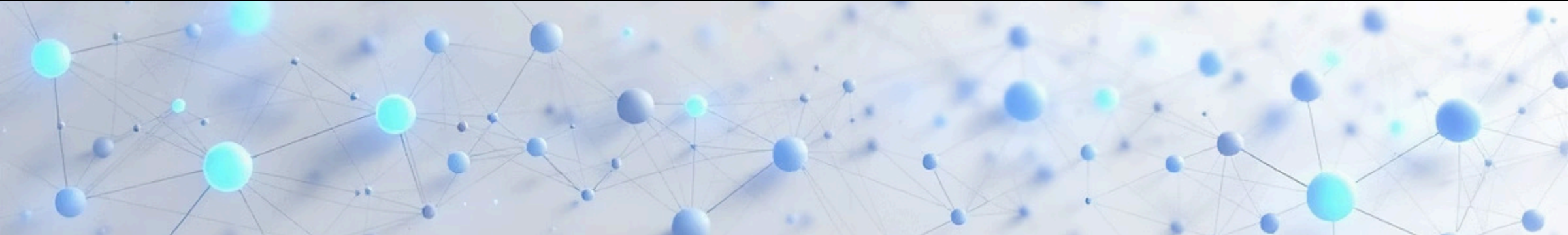
Units used by NLP models

Example:

- "The"
- "cat"
- "jumped"

Key point:

Models process tokens, not raw text.



Tokenization

Tokenization = splitting text into tokens.

Example

Input: "The cat jumped"

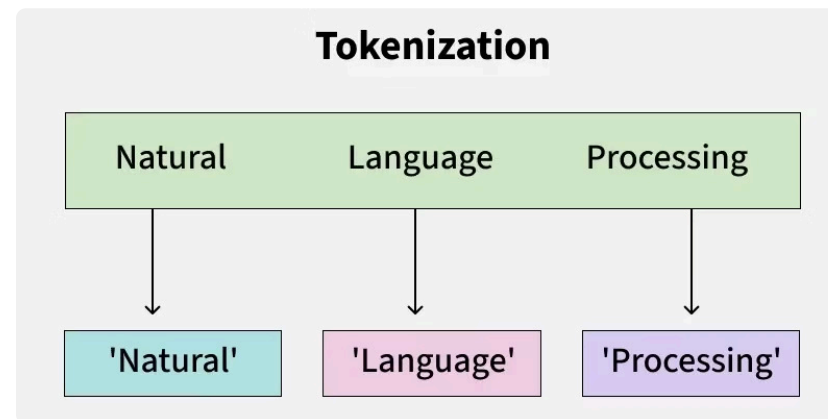
Output: ["The", "cat", "jumped"]

Different approaches:

- word tokenization
- character tokenization
- subword tokenization

Why it matters:

- controls vocabulary size
- helps handle rare words





Byte Pair Encoding (BPE)

BPE is a subword tokenization algorithm.

Idea:

01	02
Start with characters	Find most frequent pairs
03	04
Merge them	Repeat

Example corpus

- low
- lower
- lowest

Possible merges:

- lo + w → low
- low + er → lower

Advantages:

- handles rare words
- smaller vocabulary
- widely used in LLMs



Language Models

A language model assigns probability to sequences.

$$P(W_1, W_2, \dots, W_n)$$

Example sentence:

The cat jumped

Model estimates

$$P(The, cat, jumped)$$



Chain Rule of Probability

Sentence probability can be decomposed.

$$P(W_1, W_2, \dots, W_n) = P(W_1)P(W_2 \mid W_1)P(W_3 \mid W_1, W_2) \dots P(W_n \mid W_1^{n-1})$$

- This means each word depends on all previous words.
- But computing this exactly is expensive.



Markov Assumption & N-grams

We approximate history using the Markov assumption.

$$P(W_n \mid W_1^{n-1}) \approx P(W_n \mid W_{n-1})$$

This leads to N-gram models.

General form:

$$P(W_n \mid W_1^{n-1}) \approx P(W_n \mid W_{n-n+1}^{n-1})$$

Examples:

Model	Context
unigram	none
bigram	previous word
trigram	previous two words

Maximum Likelihood Estimation

We estimate probabilities from observed counts.

Goal:

$$\hat{\theta} = \operatorname{argmax}_{\theta} P(X \mid \theta)$$

For bigrams:

$$P(W_n \mid W_{n-1}) = C(W_{n-1}W_n) / C(W_{n-1})$$

Example:

$C(\text{cat jumped}) = 5$

$C(\text{cat}) = 20$

$$P(\text{jumped} \mid \text{cat}) = 5/20$$



Evaluating Language Models

Perplexity

Perplexity measures model quality.

$$PP(W) = P(W)^{(-1/N)}$$

Equivalent form:

$$PP(W) = 1/(P(W))^{(1/N)}$$

Interpretation:

- lower perplexity → better model
- measures how 'surprised' the model is

Cross Validation

Used to evaluate models reliably.

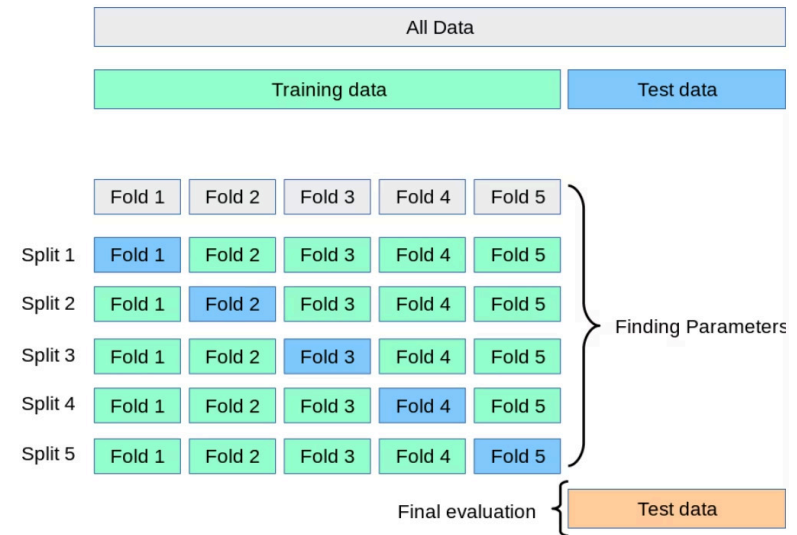
K-fold cross validation

Steps:

- Split dataset into K partitions
- Train on K-1 partitions
- Test on remaining partition
- Repeat for all partitions
- Average results

Benefit:

- improves generalizability
- reduces overfitting



Word Embeddings

Embeddings represent words as vectors.

Example:

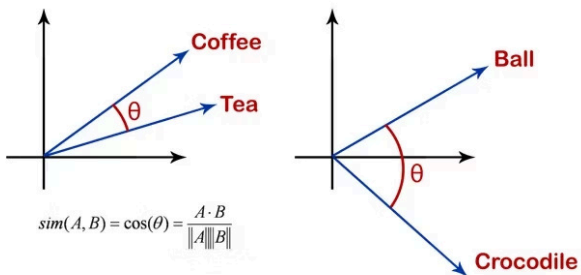
Coffee $\rightarrow$ [0.2, -0.4, 0.9, ...]

Semantic similarity can be measured using:

Dot product

$$v \cdot w = \sum v_i w_i$$

Cosine similarity





Large Language Models

Modern models generate text token by token.

These are autoregressive models.

Next token selection:

Greedy decoding

$$W_t = \operatorname{argmax} P(W \mid W < t)$$

Or random sampling from distribution:

$$X \sim \operatorname{Multinomial}(P_1, \dots, P_k)$$

Softmax converts logits into probabilities.

Temperature scaling:

$$y = \operatorname{softmax}(u/\tau)$$

- $\tau \rightarrow 0$: deterministic
- $\tau \rightarrow 1$: more random



Training LLMs

Typical training stages:

01

Pretraining

learn grammar and language patterns

02

Instruction tuning

learn to follow instructions

03

Alignment

adjust behavior to human preferences



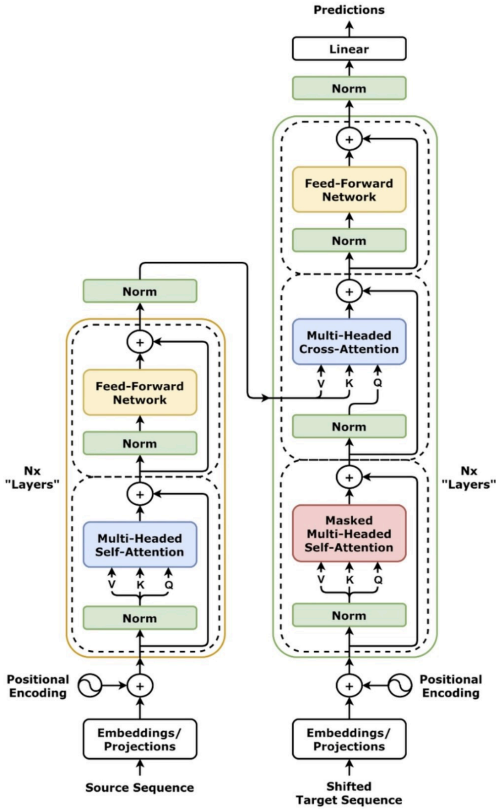
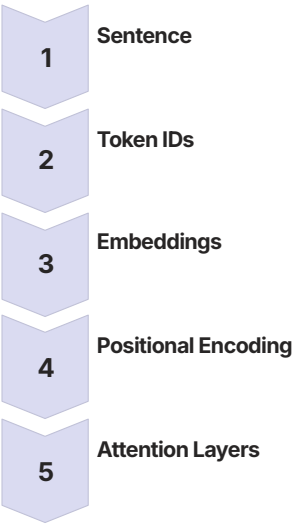
Transformers

Transformers are the architecture behind modern LLMs.

Advantages:

- parallel computation
- captures long-range dependencies
- context-aware representations

Pipeline:





Attention Mechanism

Attention allows models to focus on relevant words.

Key components:

- Query (Q)
- Key (K)
- Value (V)

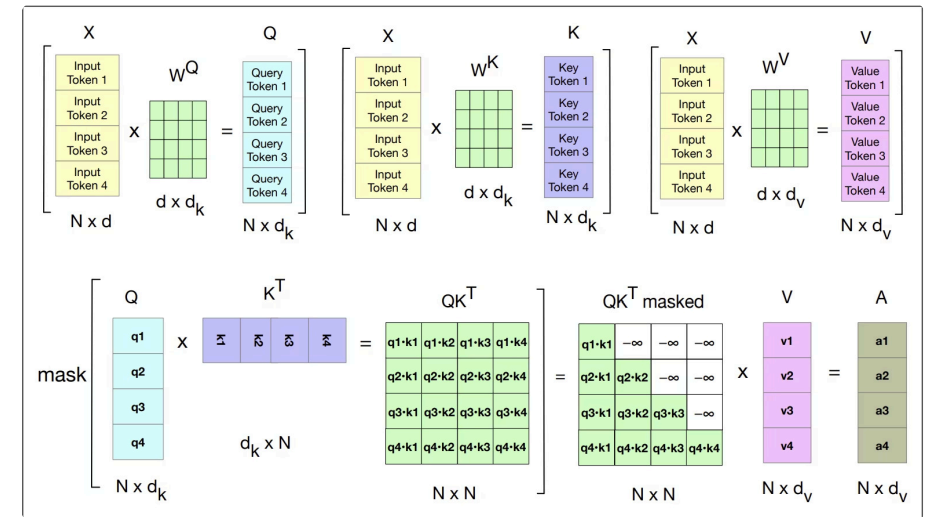
Attention computation:

$$\text{softmax}(QK^T / \sqrt{d})V$$

Meaning:

- compare tokens using QK^T
- normalize with softmax
- combine information with V

- **Linear Projections:** The input matrix X is multiplied by learned weight matrices W^Q , W^K , and W^V to produce the Query (Q), Key (K), and Value (V) matrices.
- **Similarity Scoring (QK^T):** The model computes the dot product between queries and keys to determine how much 'attention' each token should pay to every other token in the sequence.
- **Causal Masking:** In autoregressive models, a mask is applied to the scores (replacing future values with $-\infty$) to ensure the model can only 'see' current and past tokens when making predictions.
- **Normalization (Softmax):** The masked scores are normalized using the softmax function, converting them into probabilities that sum to 1 for each token.
- **Contextual Representation (A):** These probabilities are multiplied by the Value matrix (V) to create the final output A , where each token's vector is now a weighted sum of all relevant information in the sequence.





Conclusion

- **Tokenization**
Converts text into model inputs
- **N-gram models**
Approximate language probabilities
- **MLE**
Estimates probabilities from data
- **Perplexity**
Evaluates language models
- **Embeddings**
Capture semantic meaning
- **Transformers**
Enable modern LLMs